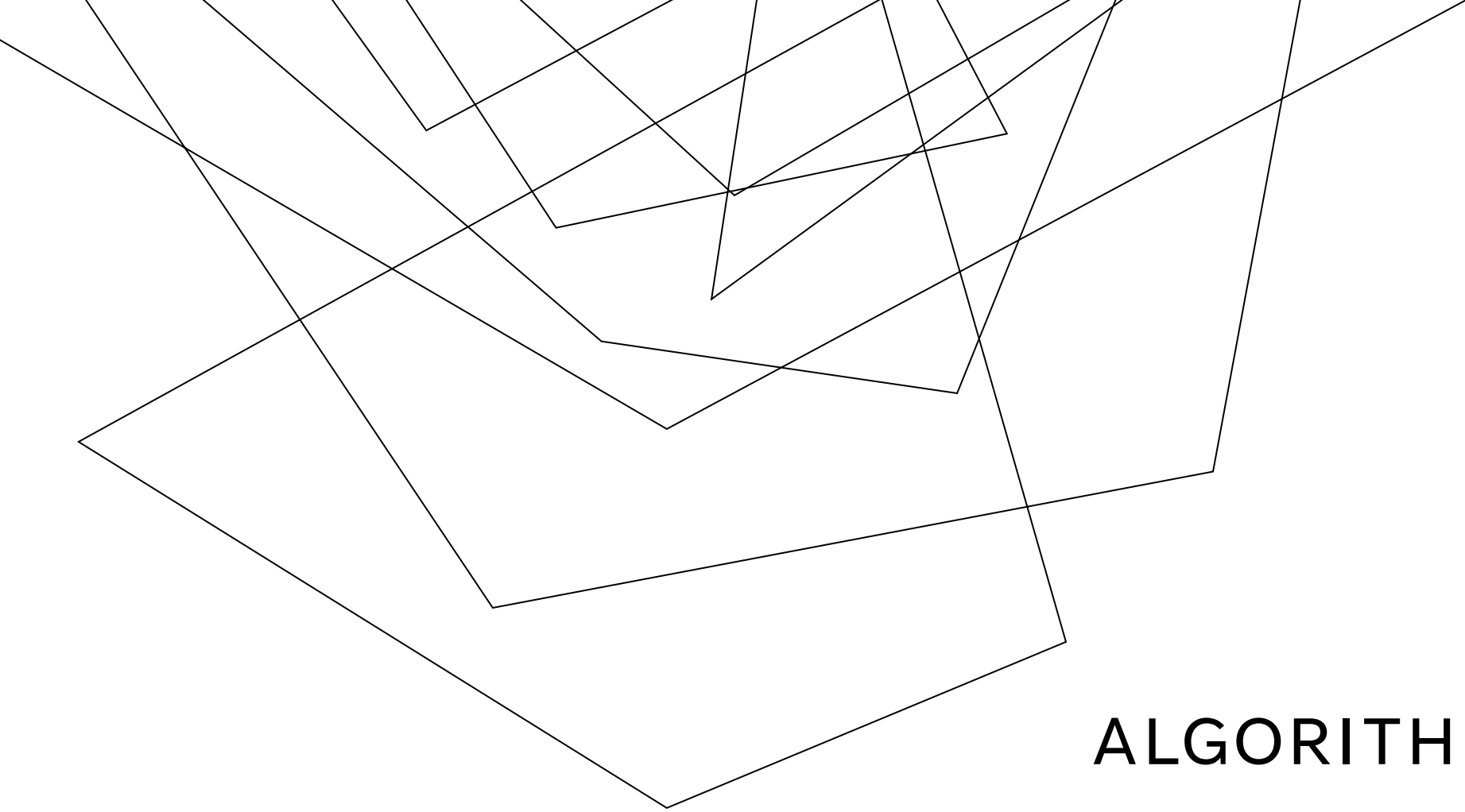


A series of thin, black, overlapping lines forming various geometric shapes like triangles and polygons, creating a complex, abstract pattern in the upper left portion of the slide.

ALGORITHMS

7. SORTING (4)





지난 시간 복습

SORTING (3)

1. Heap (data structure)

2. Heapsort





목차

목차

1. Comparison sorting
2. Counting sort
3. Radix sort



Two thin, dark gray lines intersect diagonally on a light gray background. One line slopes downwards from left to right, and the other slopes upwards from left to right.

COMPARISON SORTING

COMPARISON SORTING

지금까지 배운 정렬 알고리즘 :

- Bubble sort
- Selection sort
- Insertion sort
- Merge sort
- Quick sort
- Heap sort

➔ 공통점 : 배열 내 원소간 대소 관계 비교 연산 ($>$ or $<$) 기반



COMPARISON SORTING

지금까지 배운 정렬 알고리즘 :

- Bubble sort
- Selection sort
- Insertion sort
- Merge sort
- Quick sort
- Heap sort

➔ 공통점 : 배열 내 원소간 대소 관계 비교 연산 ($>$ or $<$) 기반

➔ 비교 연산 없이 $\Theta(n \log n)$ 보다 더 빨리 정렬 할 수 있을까?



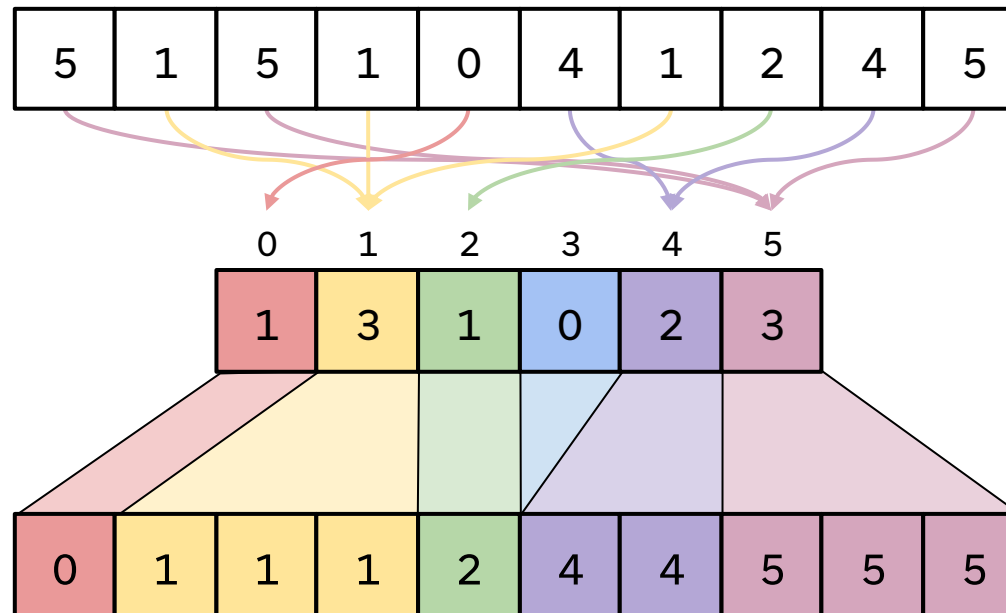
An abstract geometric design featuring two thin, dark lines that intersect on a light gray background. One line is oriented diagonally from the top-left towards the bottom-right, while the other is oriented from the top-right towards the bottom-left. The intersection point is located in the upper-left quadrant of the image.

COUNTING SORT

COUNTING SORT

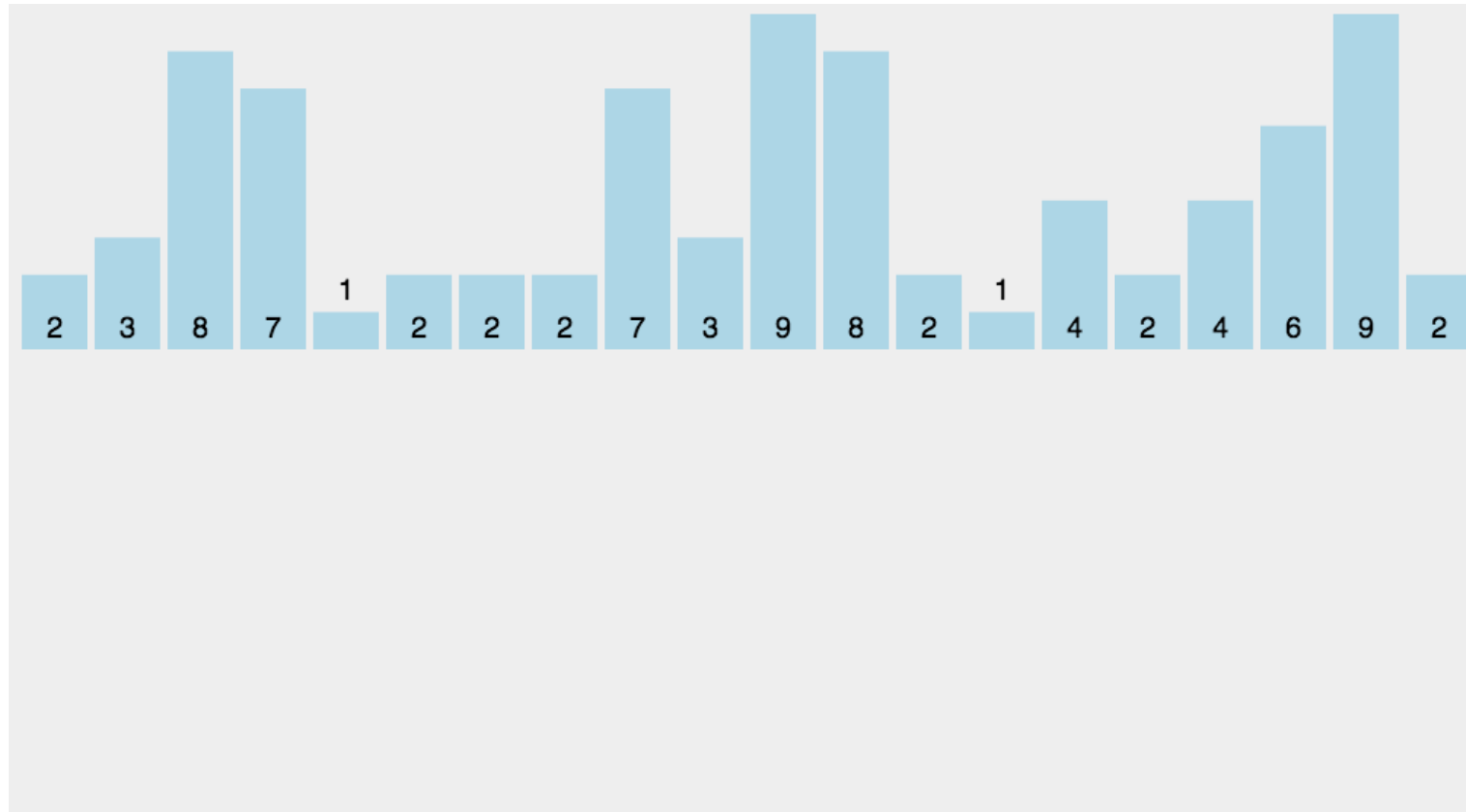
Intuition : (상대적 대소 관계 대신) 배열 내 원소들 값의 분포를 고려
각 값 개수를 세면, 값마다 들어갈 위치가 정해짐

조건 : 배열 내 값들이 r 이하인 음이 아닌 정수



COUNTING SORT

Animation



COUNTING SORT

arr

5	1	5	1	0	4	1	2	4	5
---	---	---	---	---	---	---	---	---	---

count

cnts

0	1	2	3	4	5
1	3	1	0	2	3

cumulate

cnts

0	1	2	3	4	5
1	4	5	5	7	10

sorted

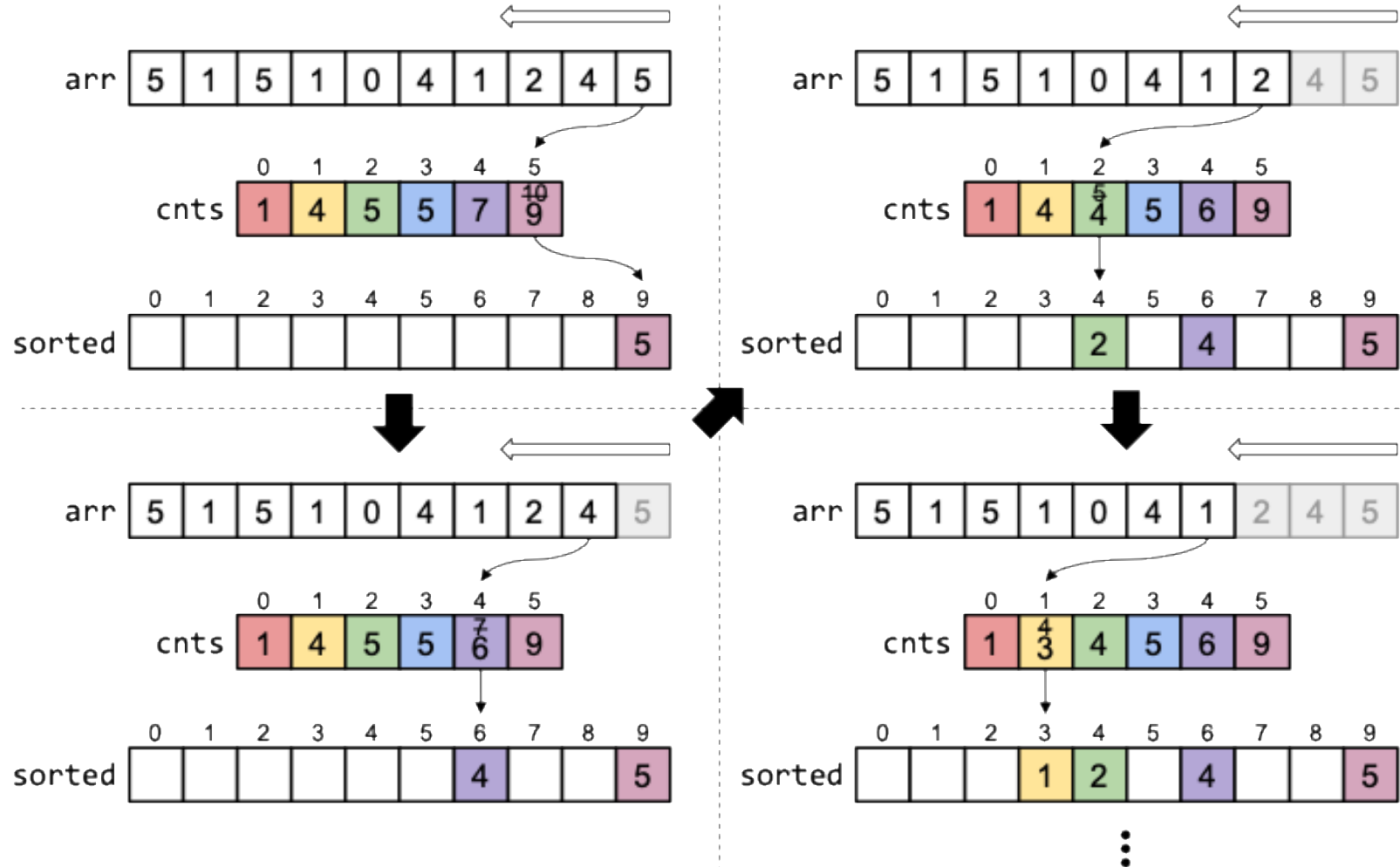
0	1	2	3	4	5	6	7	8	9	10
0	1	1	1	2	4	4	5	5	5	

각 누적값은 해당 요소가 들어갈
(마지막+1) 위치를 가리킴

```
def counting_sort(arr, r):  
    n = len(arr)  
    cnts = [0] * r  
  
    for x in arr: # count  
        cnts[x] += 1  
  
    for i in range(1, r): # cumulate  
        cnts[i] += cnts[i-1]  
  
    sorted = [0] * n  
    for x in reversed(arr): # sort  
        cnts[x] -= 1  
        sorted[cnts[x]] = x  
  
    arr[:] = sorted
```



COUNTING SORT



COUNTING SORT

Complexity analysis

- $r < n$:
- $r \gg n$:

```
def counting_sort(arr, r):  
    n = len(arr)  
    cnts = [0] * r  
  
    for x in arr: # count ←  $\Theta(n)$   
        cnts[x] += 1  
  
    for i in range(1, r): # cumulate ←  $\Theta(r)$   
        cnts[i] += cnts[i-1]  
  
    sorted = [0] * n  
    for x in reversed(arr): # sort ←  $\Theta(n)$   
        cnts[x] -= 1  
        sorted[cnts[x]] = x  
  
    arr[:] = sorted ←  $\Theta(n)$ 
```



An abstract graphic design featuring two thin, dark gray lines that intersect on a light gray background. One line is oriented diagonally from the top-left towards the bottom-right, while the other is oriented from the top-right towards the bottom-left. The intersection point is located in the upper-left quadrant of the image. To the right of this intersection, the words "RADIX SORT" are displayed in a bold, black, sans-serif typeface.

RADIX SORT

RADIX SORT

Radix?

전체 | 단어·속어 | 뜻풀이 | 예문 | 영영사전



+ 단어장 저장

ra·dix

1. 어근(語根); 근; 기(基), 기수(基數); (뇌·신경 등의) 뿌리
2. 뿌리(root)

발음 미국·영국 [réidiks]



등아출판

YBM

교학사

슈프림



명사 | 영영사전

명사 (pl. ~es, ra·di·ces[rædəsi:z, réi-|réi-])

1. [언어] 어근(語根); [수학] 근; 기(基), 기수(基數); [해부] (뇌·신경 등의) 뿌리
2. 식물
뿌리(root)
3. 철학
근원



RADIX SORT

Radix?

기수란 무엇인가요?

기수는 기술 및 컴퓨팅의 맥락에서 숫자 체계의 기초를 의미합니다. 기수는 위치수 체계에서 숫자를 표현하는 데 사용되는 고유한 숫자(0 포함)의 수입니다. 예를 들어 2진수의 기수(기저 2)는 두 자리(0과 1)를 사용하기 때문에 2이고, 10진수의 기수(기저 10)는 열 자리(0~9)를 사용하기 때문에 10입니다.

기수는 컴퓨터 과학의 데이터 구조와 관련이 있나요?

예, 기수는 컴퓨터 과학의 특정 데이터 구조 및 알고리즘과 관련이 있습니다. 예를 들어 기수 정렬 알고리즘은 동일한 위치와 값을 공유하는 숫자를 그룹화하여 정수 키로 데이터를 정렬하는 비비교 정렬 알고리즘입니다. 이 알고리즘은 기수를 기본으로 사용하여 숫자를 정렬합니다.

표준 기수가 아닌 다른 기수를 숫자 체계에 사용할 수 있나요?

예. 가장 많이 사용되는 기수는 10(10진수), 2(2진수), 8(8진수), 16(16진수)이지만 기술적으로는 모든 양의 정수를 기수로 사용할 수 있습니다. 이는 이론적인 컴퓨터 과학이나 다른 기수가 더 효율적일 수 있는 특정 애플리케이션에서 종종 수행됩니다.

<https://www.lenovo.com/kr/ko/glossary/what-is-radix/>



RADIX SORT

Intuition : $r \gg n$ 일 때 counting sort가 비효율적
 r 이 큰 수일 경우, 자릿수 단위로 쪼개어 생각

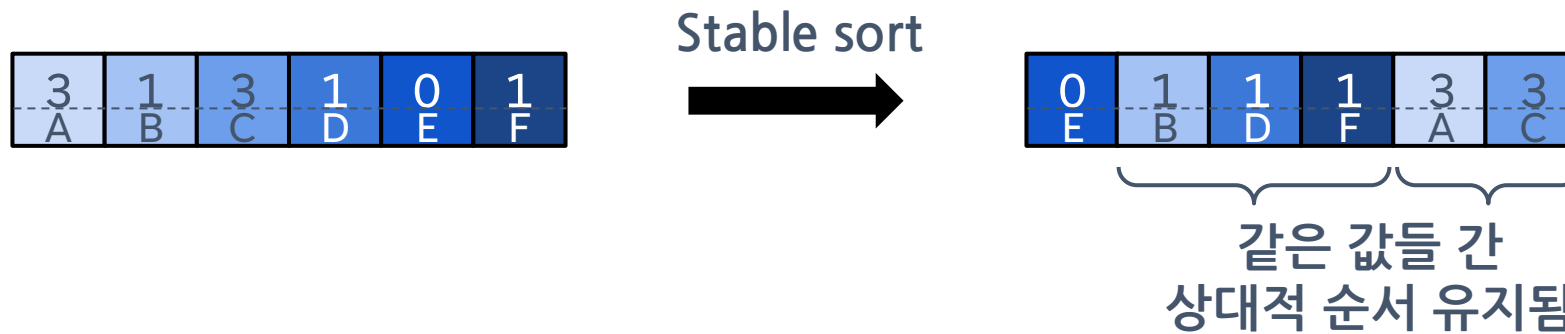
조건 : 원소들이 모두 k 자릿수 이하인 음이 아닌 정수

- LSD (Least significant digit) : 제일 낮은 자릿수
- MSD (Most significant digit) : 제일 높은 자릿수



RADIX SORT

- Key idea : 낮은 자릿수(LSD) 부터 높은 자릿수(MSD) 순서대로
자릿수를 기준으로 안정 정렬(Stable sort) 수행
- Stable sort: 같은 값들간 상대적인 순서를 유지하며 수행하는 정렬
 - Bubble sort, Insertion sort, Merge sort, Counting sort 등



RADIX SORT

0123
2154
0222
0004
0283
1560
1061
2150



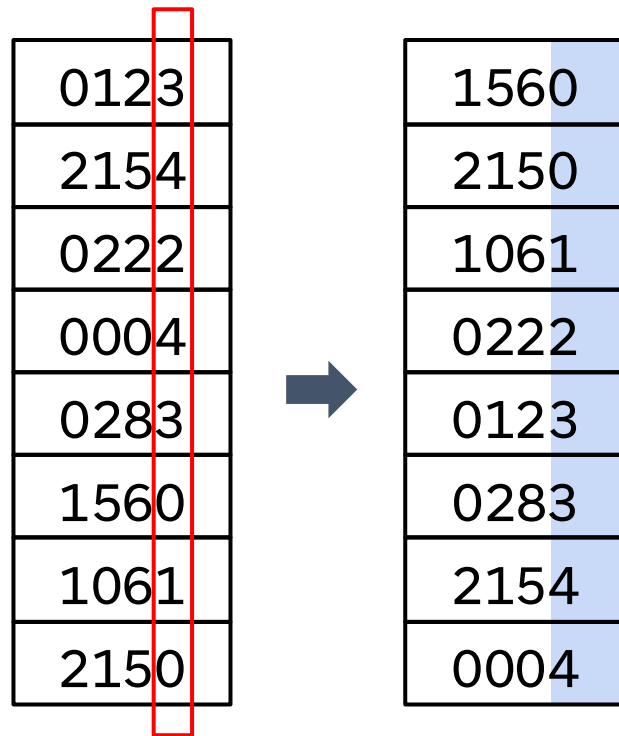
RADIX SORT

0123
2154
0222
0004
0283
1560
1061
2150

첫째 자릿수로
안정 정렬



RADIX SORT



첫째 자릿수로
안정 정렬



RADIX SORT

0123	1560
2154	2150
0222	1061
0004	0222
0283	0123
1560	0283
1061	2154
2150	0004

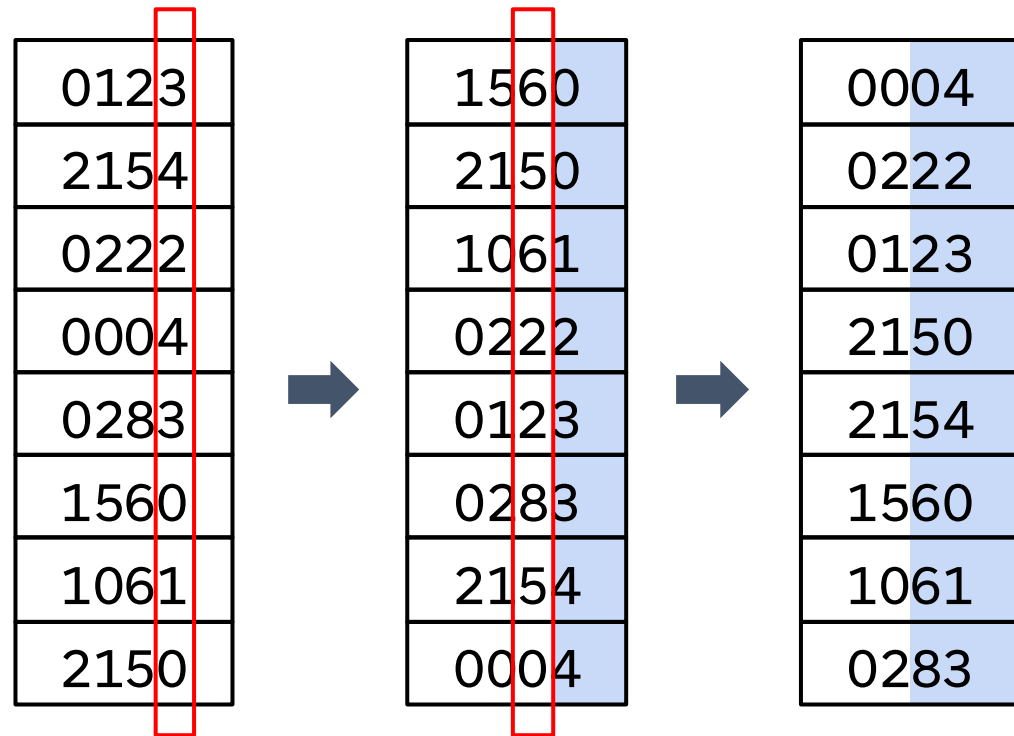


첫째 자릿수로
안정 정렬

둘째 자릿수로
안정 정렬



RADIX SORT

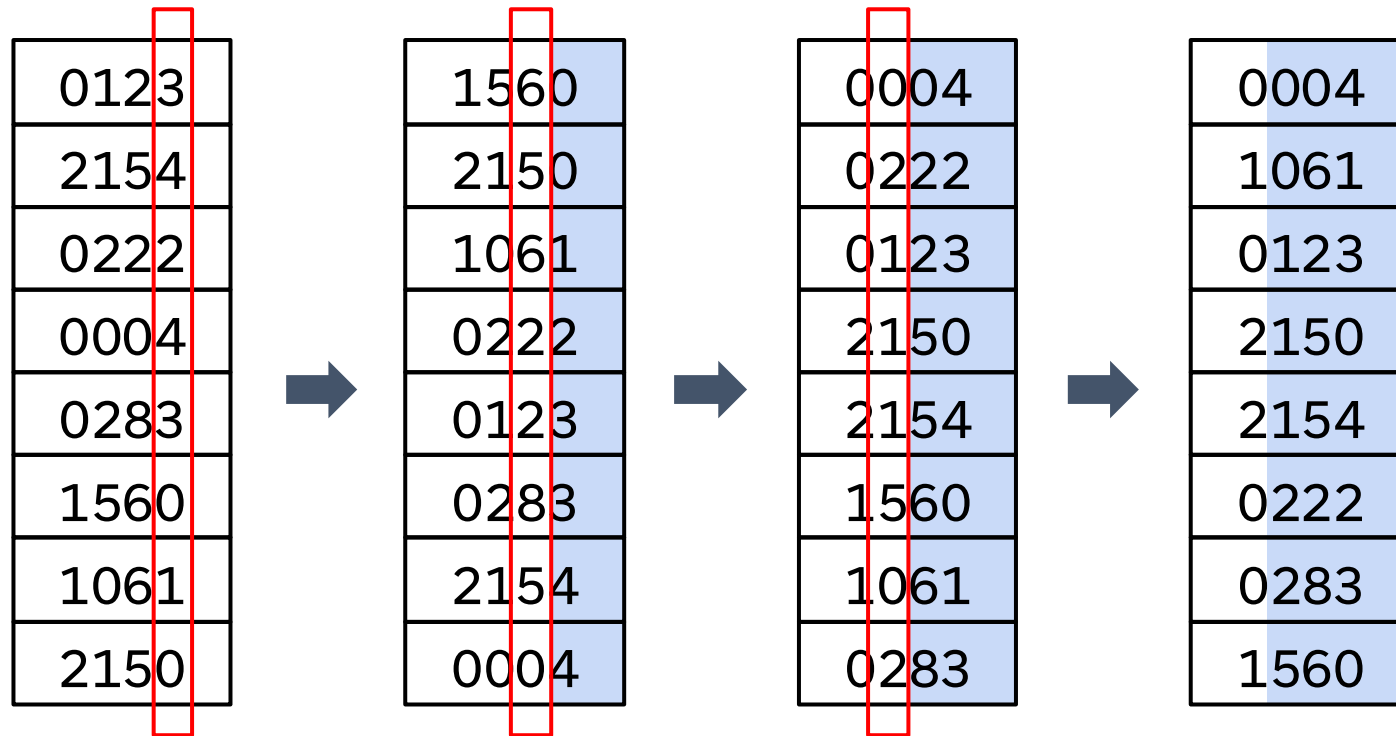


첫째 자릿수로
안정 정렬

둘째 자릿수로
안정 정렬



RADIX SORT



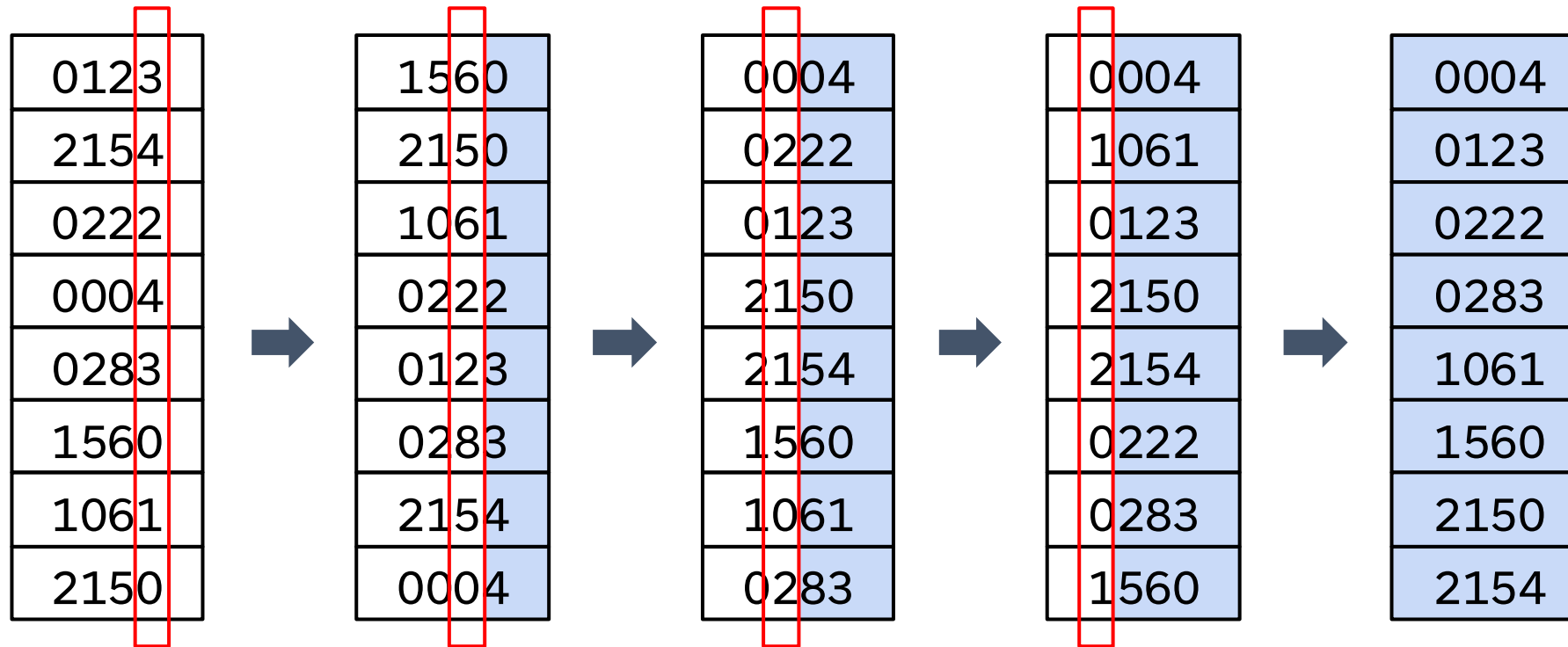
첫째 자릿수로
안정 정렬

둘째 자릿수로
안정 정렬

셋째 자릿수로
안정 정렬



RADIX SORT



첫째 자릿수로
안정 정렬

둘째 자릿수로
안정 정렬

셋째 자릿수로
안정 정렬

넷째 자릿수로
안정 정렬



RADIX SORT

Complexity analysis

- n : 주어진 값의 수, k : 총 자릿수(# digits), r : 기수(radix)
- $\Theta(r + n)$ 인 counting sort를 k 번 반복

- Worst case : 배열 내 원소 값들이 모두 다른 경우

$$k = \log_r n$$



An abstract graphic on a light gray background. Two thin, dark gray lines intersect. One line is nearly vertical, starting from the top center and extending towards the bottom right. The other line is nearly horizontal, starting from the top left and extending towards the right. The intersection point is located to the left of the word 'SUMMARY'.

SUMMARY

SUMMARY

1. Comparison sorting
2. Counting sort
3. Radix sort



NEXT LECTURE

Selection



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They cross each other in the upper-left quadrant of the image.

QUESTIONS?